Projet de complexité algorithmique

Détection de communautés dans les graphes

UTBM - IT4A - A2026

À rendre avant le 31 octobre 2026

Présentation

La détection de communautés consiste, étant donné un graphe, à partitionner ses sommets en groupes denses en interne et faiblement connectés entre eux (figure 1). Ce problème apparaît naturellement dès que l'on étudie des réseaux réels : réseaux sociaux, réseaux de citations, réseaux biologiques,... Il est aujourd'hui au cœur de nombreuses applications, de la recommandation de produits à l'analyse de la propagation de l'information.

Sa formulation exacte, qui consiste à maximiser la modularité d'une partition, est un problème NP-difficile. En pratique, on a donc recours à des heuristiques, dont certaines sont quasi-linéaires et passent à l'échelle sur des graphes de plusieurs millions de sommets.

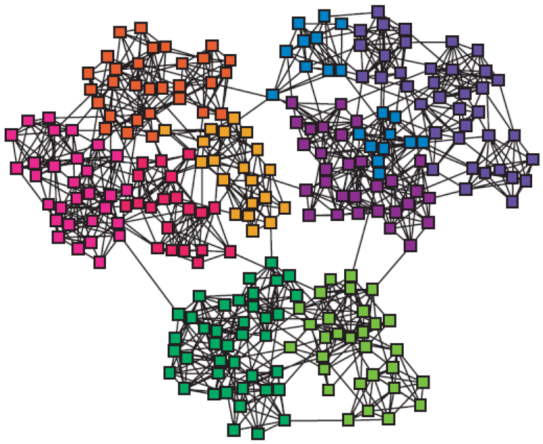


FIGURE 1 – Exemple de partition d'un graphe en communautés. Les sommets d'une même communauté sont denses en interne et peu connectés au reste du graphe. Source `https://cedric.cnam.fr/vertigo/Cours/RCP216/index.html`

Dans ce projet, vous implémenterez plusieurs algorithmes de détection de communautés, vous les comparerez sur des graphes synthétiques et réels, et vous analyserez les compromis entre qualité de la partition et temps de calcul.

Modalités de rendu

Le projet est à réaliser en binôme. Le rendu se fait par l'intermédiaire d'un seul notebook Jupyter et de fichiers python. Le notebook contiendra le code dans des cellules `code` et le texte dans des cellules `markdown`.

Un exemple de notebook est fourni avec les sujets. Ce notebook constitue un patron : il propose une structure de sections, quelques rappels de complexité et un squelette de code utilitaire.

Vous êtes libres de le suivre tel quel, de le réorganiser ou de repartir d'un notebook vierge, tant que le rendu respecte les consignes de *nommage* et contient les éléments attendus. Le notebook rendu doit être exécutable de bout en bout par l'évaluateur, dans un environnement Python standard, sans intervention manuelle. L'évaluateur exécutera le notebook tel qu'il est rendu. Si cette exécution échoue, le projet sera fortement pénalisé.

Lorsque le code devient conséquent (classes, fonctions utilitaires, générateurs de graphes, visualisations), il doit être placé dans un ou plusieurs fichiers `.py` importés depuis le notebook. Le nom des fichiers à rendre doit être de la forme :

`projet_NOM1_Prenom1_NOM2_Prenom2.ipynb`
`[projet1.py]`...`[projetN.py]`s

Usage de l'intelligence artificielle

L'usage d'outils d'IA générative est *autorisé et encouragé* pour produire du code, dégager des pistes d'analyse ou reformuler certaines parties. Cependant, tout passage produit ou assisté par une IA doit être signalé explicitement dans le notebook, dans une cellule `markdown` dédiée, en précisant l'outil utilisé et la nature de l'assistance (génération de code, suggestion d'analyse, reformulation). Le code fourni par une IA doit être compris, testé et validé ; une simple reproduction sans compréhension sera fortement **pénalisée** si elle est détectée.

Algorithmes à implémenter et étudier

1. Un générateur de graphes synthétiques à communautés plantées [1], permettant de contrôler le nombre de sommets, le nombre de communautés, la densité interne et la densité externe. Le modèle de Girvan-Newman ou une variante simplifiée du modèle LFR conviennent.
2. L'algorithme de Girvan-Newman.
3. L'algorithme de Louvain.
4. L'algorithme de propagation d'étiquettes, dans sa version simple puis dans une version stabilisée.
5. Une recherche exhaustive ("force brute") ou une méthode exacte sur de très petites instances, servant de référence pour évaluer la qualité des heuristiques.

Vous pouvez utiliser les implémentations disponibles dans des bibliothèques telles que `NetworkX`, ou écrire vous-mêmes les algorithmes. Ce choix n'a aucune incidence sur l'évaluation. En revanche, la compréhension de chaque algorithme est exigée : pour chacun d'eux, vous devez expliquer dans une cellule `markdown` dédiée son principe et illustrer son fonctionnement à la main sur une petite instance, par exemple en détaillant une itération complète avec les calculs intermédiaires.

Vous utiliserez des graphes de tailles variées :

— petites instances (10 à 30 sommets), pour lesquelles une méthode exacte est envisageable ;

— instances moyennes (100 à 1000 sommets), pour comparer les heuristiques entre elles ;

— instances grandes (plus de 1000 sommets), pour observer le passage à l'échelle ;

— le graphe réel *Cora*, qui sera utilisé par tous les binômes. Il s'agit d'un réseau de citations scientifiques comportant 2 708 sommets, 5 429 arêtes et 7 classes thématiques, ce qui en fait un jeu de données de taille moyenne particulièrement adapté à la comparaison d'algorithmes de détection de communautés. Le fichier est téléchargeable à l'adresse

1. « Communautés plantées » traduit l'anglais "planted communities". L'idée est que l'on ne se contente pas de générer un graphe aléatoire quelconque : on y injecte volontairement une structure de communautés, dont on connaît à l'avance la composition exacte. On dit que l'on « plante » ces communautés dans le graphe, comme on plante une graine.

`https://linqs-data.soe.ucsc.edu/public/lbc/cora.tgz`. Le notebook fourni contient le code de téléchargement et de chargement.

Attendus

Le but du projet est de :

— comprendre la nature NP-difficile de la maximisation exacte de la modularité ;

— expliquer le principe de chaque algorithme et illustrer une itération à la main ;

— analyser empiriquement les performances en fonction de la taille et de la structure des graphes ;

— produire des visualisations claires des graphes et des partitions obtenues ;

— porter un regard critique sur les compromis temps de calcul et qualité de la solution.

Pour chaque algorithme et chaque instance, les métriques à évaluer sont :

— le nombre d'opérations ou d'itérations, à relier à la complexité théorique ;

— le temps d'exécution moyen, à relier à la complexité empirique ;

— la qualité de la partition, mesurée par la modularité, et par l'indice de Rand ajusté ou l'information mutuelle normalisée lorsque des communautés de référence sont disponibles ;

— l'utilisation mémoire, en option.

Barème sur 100 points

La partie code ne rapporte aucun point en elle-même : du code disponible en ligne ou produit par une IA peut être utilisé, à condition d'être validé et compris (voir la section « Usage de l'intelligence artificielle »).

Le notebook sera exécuté par l'évaluateur. Toute erreur d'exécution empêchant l'évaluation du projet entraîne une pénalité pouvant aller jusqu'à la note **nulle**.

La qualité générale du notebook (lisibilité, commentaires, illustrations) compte pour **10 points**. Une brève présentation de la théorie de la complexité en début de notebook compte pour **10 points**.

La description de chaque algorithme et l'étude théorique de sa complexité comptent pour **15 points**. L'illustration manuelle d'une itération pour chaque algorithme compte pour **15 points** : c'est le principal indicateur de compréhension, et il sera évalué comme tel.

La comparaison des algorithmes selon les différents critères compte pour **40 points** : temps d'exécution en lien avec la complexité, qualité de la partition pour les algorithmes approchés, compromis temps et qualité. Les comparaisons seront illustrées par des courbes d'évolution lorsque la taille des graphes ou le nombre d'instances augmente.

Enfin, une conclusion générale et un compte rendu réflexif sur les apports du projet à votre compréhension de l'informatique comptent pour **10 points**.

Références

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.

[2] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Physical Review E*, vol. 69, no. 2, p. 026113, Feb. 2004. DOI : `10.1103/PhysRevE.69.026113`.

[3] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics : Theory and Experiment*, P10008, 2008.

[4] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical Review E*, vol. 76, no. 3, p. 036106, Sep. 2007.

[5] S. Fortunato, "Community detection in graphs," *Physics Reports*, vol. 486, no. 3-5, pp. 75–174, 2010.

[6] J. Leskovec and R. Sosič, *SNAP : Stanford Network Analysis Project*, `http://snap.stanford.edu`, accessed Sep. 2026.

[7] J. Yang and J. Leskovec, "Defining and evaluating network communities based on ground-truth," *Knowledge and Information Systems*, vol. 45, no. 3, pp. 589–616, 2015.